

# Object Detection

---

## Object Detection

1. Enable the Board's Best Performance
    - 1.1 Enable MAX Power Mode
    - 1.2 Enable Jetson Clocks
  2. Object Prediction: Image
    - Effect Preview
  3. Object Prediction: Video
    - Effect Preview
  4. Object Prediction: Real-time Detection
    - 4.1 USB Camera
      - Effect Preview
    - 4.2 CSI Camera
      - Effect Preview
- [Reference](#)

Demonstrates **Ultralytics**: Object Detection effects on images, videos, and real-time detection using Python.

## 1. Enable the Board's Best Performance

---

### 1.1 Enable MAX Power Mode

Enabling MAX Power Mode on the Jetson ensures all CPU and GPU cores are turned on:

```
sudo nvpmodel -m 2
```

### 1.2 Enable Jetson Clocks

Enabling Jetson Clocks ensures all CPU and GPU cores run at maximum frequency:

```
sudo jetson_clocks
```

## 2. Object Prediction: Image

---

Use yolo11n.pt to predict images that come with the ultralytics project.

Enter the code folder:

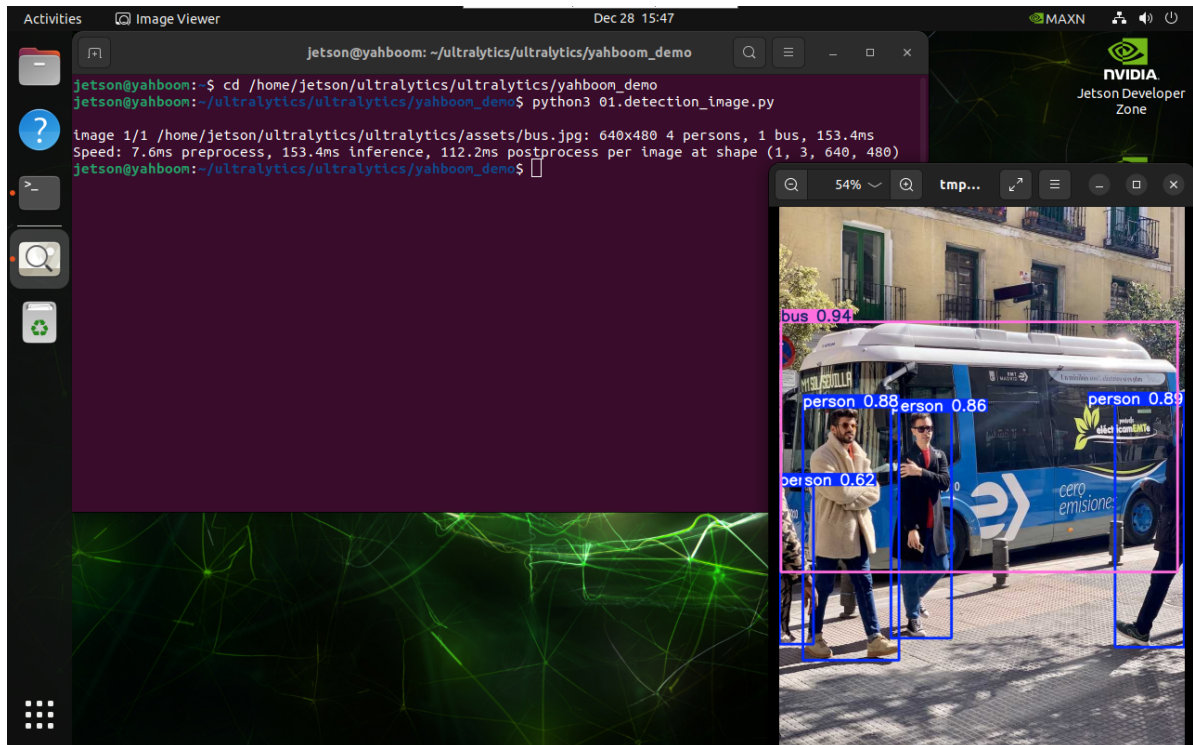
```
cd /home/jetson/ultralytics/ultralytics/yahboom_demo
```

Run the code:

```
python3 01.detection_image.py
```

## Effect Preview

The location of images output by YOLO recognition: /home/jetson/ultralytics/ultralytics/output/



Example code:

```
from ultralytics import YOLO

# Load a model
model = YOLO("/home/jetson/ultralytics/ultralytics/yolo11n.pt")

# Run batched inference on a list of images
results = model("/home/jetson/ultralytics/ultralytics/assets/bus.jpg") # return
a list of Results objects

# Process results list
for result in results:
    boxes = result.boxes # Boxes object for bounding box outputs
    # masks = result.masks # Masks object for segmentation masks outputs
    # keypoints = result.keypoints # Keypoints object for pose outputs
    # probs = result.probs # Probs object for classification outputs
    # obb = result.obb # Oriented boxes object for OBB outputs
    result.show() # display to screen

    result.save(filename="/home/jetson/ultralytics/ultralytics/output/bus_output.jpg") # save to disk
```

## 3. Object Prediction: Video

Use yolo11n.pt to predict a video under the ultralytics project (a video not included with ultralytics).

Enter the code folder:

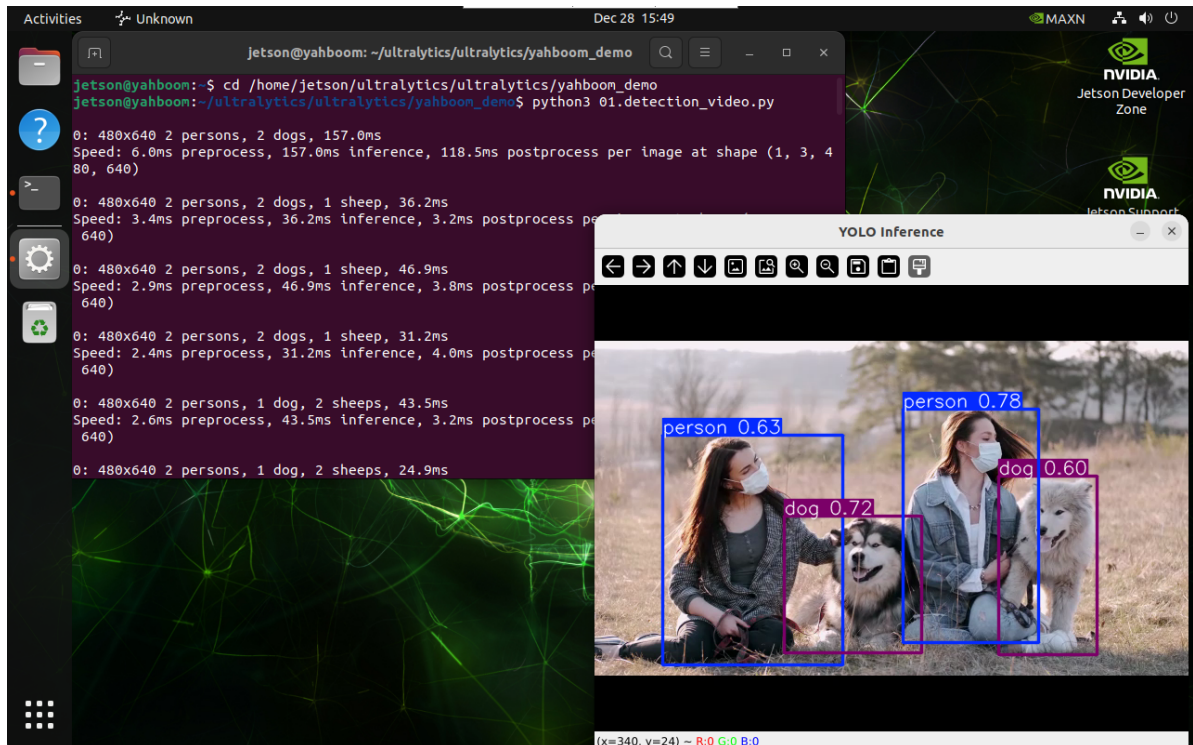
```
cd /home/jetson/ultralytics/ultralytics/yahboom_demo
```

Run the code:

```
python3 01.detection_video.py
```

## Effect Preview

The location of videos output by YOLO recognition: /home/jetson/ultralytics/ultralytics/output/



Example code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/home/jetson/ultralytics/ultralytics/yolo11n.pt")

# Open the video file
video_path = "/home/jetson/ultralytics/ultralytics/videos/people_animals.mp4"
cap = cv2.VideoCapture(video_path)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a Videowriter object to output the processed video
output_path =
"/home/jetson/ultralytics/ultralytics/output/01.people_animals_output.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()
```

```

if success:
    # Run YOLO inference on the frame
    results = model(frame)

    # Visualize the results on the frame
    annotated_frame = results[0].plot()

    # Write the annotated frame to the output video file
    out.write(annotated_frame)

    # Display the annotated frame
    cv2.imshow("YOLO Inference", annotated_frame)

    # Break the loop if 'q' is pressed
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break
else:
    # Break the loop if the end of the video is reached
    break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

## 4. Object Prediction: Real-time Detection

### 4.1 USB Camera

Use yolo11n.pt to predict the USB camera feed.

Enter the code folder:

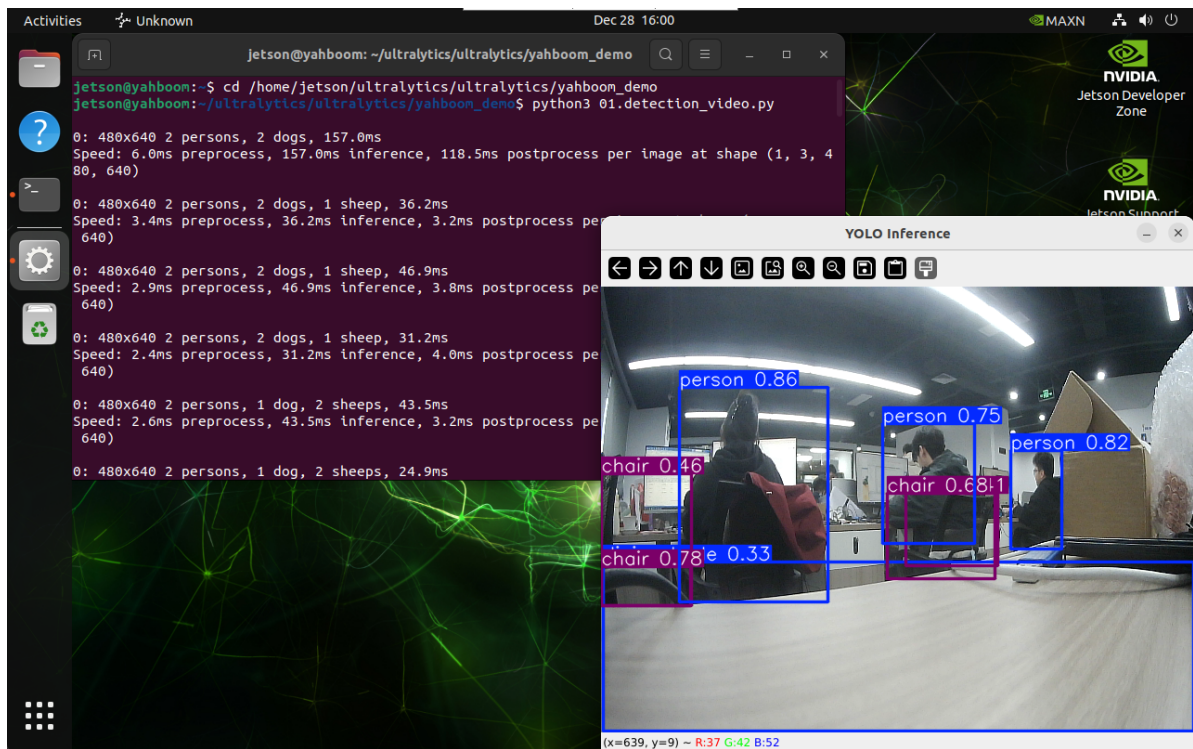
```
cd /home/jetson/ultralytics/ultralytics/yahboom_demo
```

Run the code: Click on the preview window and press the q key to terminate the program!

```
python3 01.detection_camera_usb.py
```

### Effect Preview

The location of videos output by YOLO recognition: /home/jetson/ultralytics/ultralytics/output/



Example code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/home/jetson/ultralytics/ultralytics/yolo11n.pt")

# Open the camera
cap = cv2.VideoCapture(0)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path =
"/home/jetson/ultralytics/ultralytics/output/01.detection_camera_usb.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
```

```

out.write(annotated_frame)

# Display the annotated frame
cv2.imshow("YOLO Inference", annotated_frame)

# Break the loop if 'q' is pressed
if cv2.waitKey(1) & 0xFF == ord("q"):
    break
else:
    # Break the loop if the end of the video is reached
    break
# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

## 4.2 CSI Camera

Use yolo11n.pt to predict the CSI camera feed.

Enter the code folder:

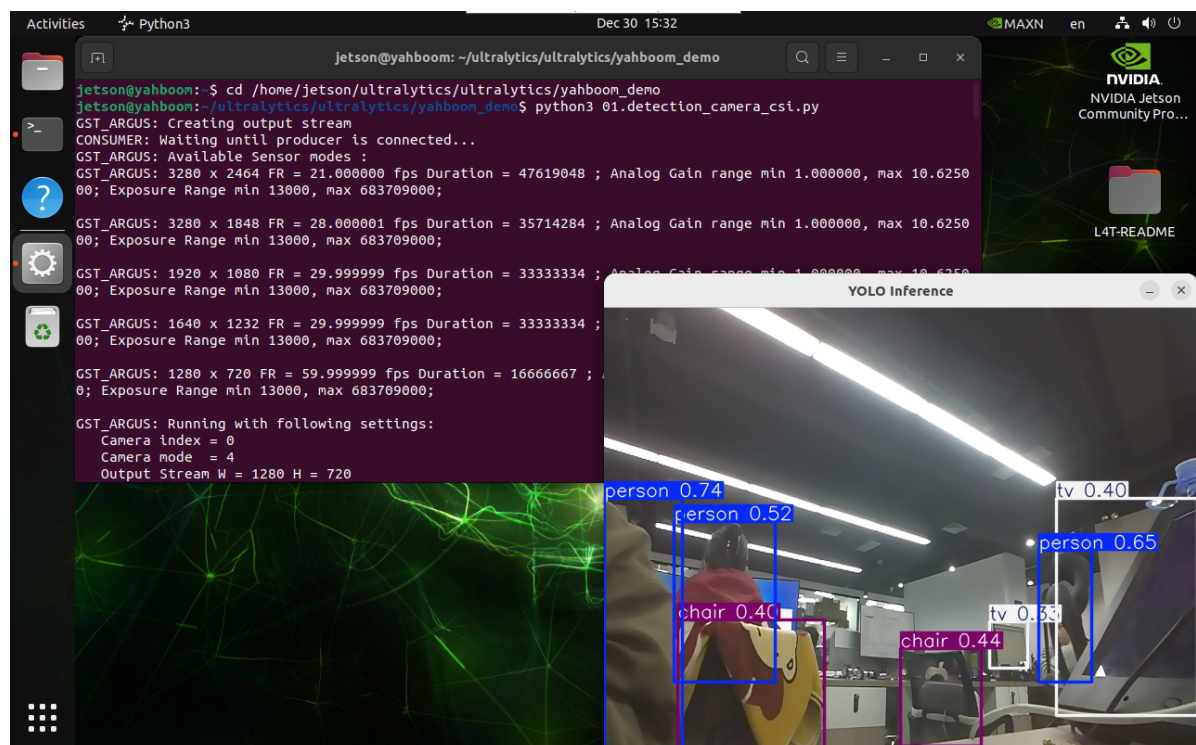
```
cd /home/jetson/ultralytics/ultralytics/yahboom_demo
```

Run the code: Click on the preview window and press the q key to terminate the program!

```
python3 01.detection_camera_csi.py
```

## Effect Preview

The location of videos output by YOLO recognition: /home/jetson/ultralytics/ultralytics/output/



Example code:

```
import cv2
```

```

from ultralytics import YOLO
from jetcam.csi_camera import CSICamera

# Load the YOLO model
model = YOLO("/home/jetson/ultralytics/ultralytics/yolo11n.pt")

# Open the camera (CSI Camera)
cap = CSICamera(capture_device=0, width=640, height=480)

# Get the video frame size and frame rate
frame_width = 640
frame_height = 480
fps = 30

# Define the codec and create a Videowriter object to output the processed video
output_path =
"/home/jetson/ultralytics/ultralytics/output/01.detection_camera_csi.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while True:
    # Read a frame from the camera
    frame = cap.read()

    if frame is not None:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if no frame is received (camera error or end of stream)
        print("No frame received, breaking the loop.")
        break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

## Reference

<https://docs.ultralytics.com/tasks/detect/>

